

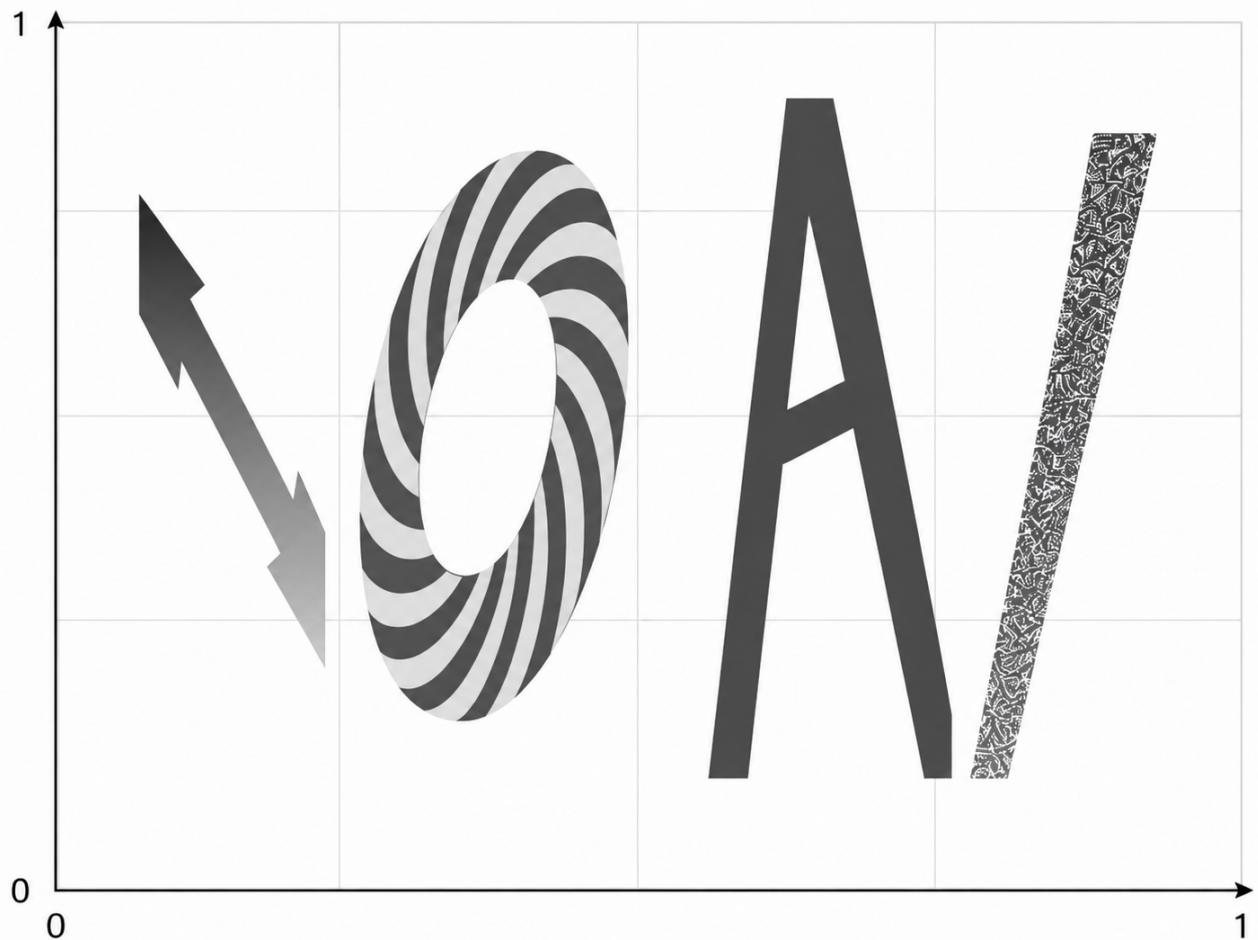
Campo IOAI

- **Límite de tiempo:** 5 minutos
- **Almacenamiento:** 5 GB
- **Tamaño de la solución:** `solution.ipynb`, `custom_model.py` ≤ 1 MB en conjunto
- **Modelos preentrenados:** ninguno — entrenar desde cero, sin internet durante la evaluación
- **Puntuación de referencia:** 31.2187

Tarea

El alcalde de Astana quiere decorar la ciudad con logotipos estilizados de la IOAI. Como estadístico, considera todo —incluido el logotipo— como una función espacial $F(x, y, \overline{W})$, donde $x, y \in [0, 1]$ representan coordenadas en un plano 2D y $\overline{W}$ es un conjunto de parámetros ocultos que definen atributos estilísticos como los colores y los ángulos de las letras.

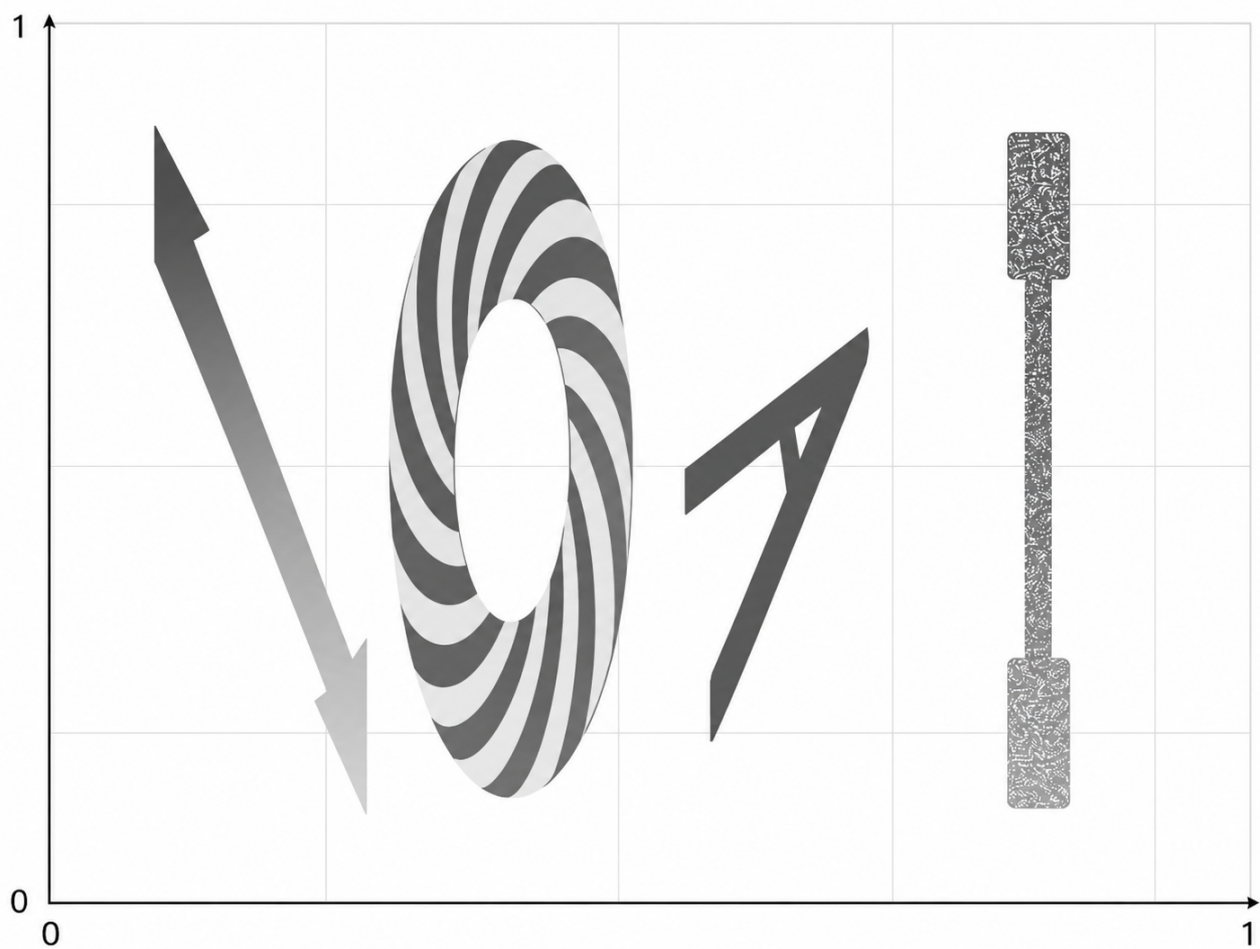
Debido a que F es demasiado compleja para expresarla como una ecuación matemática explícita, su tarea es entrenar una red neuronal para aproximarla. La red generará un valor de **campo IOAI** para cualquier par de coordenadas (x, y) , produciendo una visualización completa del logotipo como mapa de calor sobre el plano. Este es un ejemplo de visualización como mapa de calor de F con ciertos parámetros ocultos específicos $\overline{W}$.

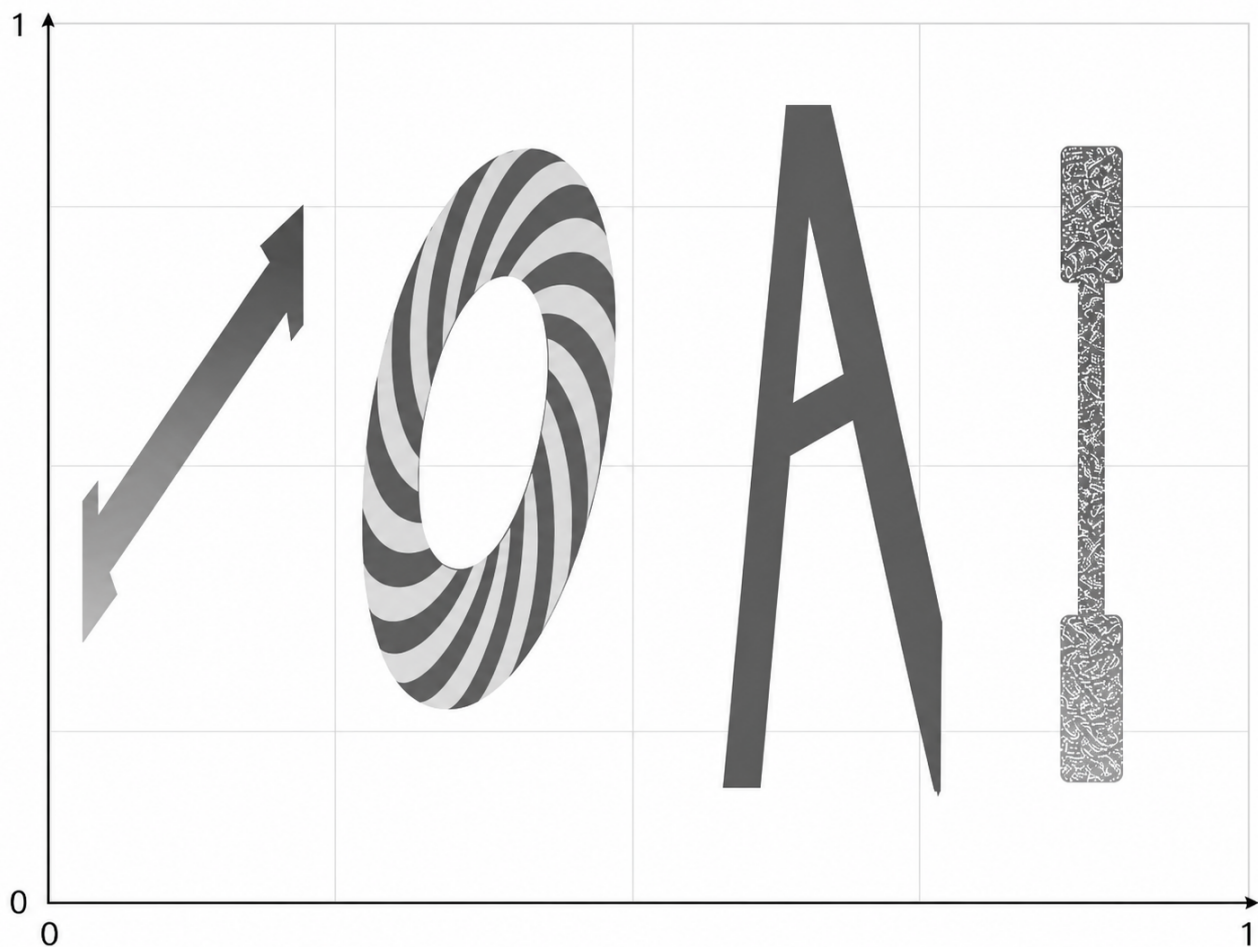


¿De qué consta el campo IOAI? De cuatro letras y el fondo.

- Los valores dentro de la primera letra $\mathbb{I}$ son muy grandes ($1e+10$ o más) y presentan un gradiente lineal
- Los valores en la letra $\mathbb{O}$ muestran un patrón espiral
- El valor dentro de la letra $\mathbb{A}$ siempre es -1
- Los valores dentro de la última letra $\mathbb{I}$ deben ser valores aleatorios del rango $[-2026, 2026]$ incluso si se evalúa dos veces el mismo punto
- Fuera de las letras, el valor siempre es cero

La función tiene parámetros ocultos $\overline{W}$, que afectan a la escala y la inclinación de las letras, junto con el rango de valores dentro de la primera letra $\mathbb{I}$. Sin embargo, las letras no se intersectarán. Estos son algunos ejemplos ilustrativos del aspecto del campo IOAI con diferentes $\overline{W}$:





Qué se proporciona:

Este problema NO contiene datasets. En su lugar, se proporciona la función generadora, que se configura mediante el archivo de configuración JSON ubicado en `data/train_config/field_config.json`.

La configuración de prueba está oculta, pero es de naturaleza similar. Su tarea es ajustar el modelo al generador proporcionado utilizando tantos datos como desee. Sus distribuciones de «entrenamiento» y «prueba» se generan a partir del mismo generador; simplemente no sabe en qué puntos (x_i, y_i) se le evaluará.

Su entrega debe constar de:

- la clase del modelo de entrenamiento guardada como `custom_model.py`. Este modelo debe heredar de la clase `torch.nn.Module` y usar únicamente imports de `torch`. Debe contener la clase `CustomModel` utilizada en el notebook `solution.ipynb`.
- el notebook `solution.ipynb`, que producirá los pesos `model.pt`

Puntuación

Para cada región, la puntuación mínima es 0 y la máxima es 1. La puntuación final se promedia entre las cinco regiones (cuatro, una por cada letra, y el fondo) y se multiplica por 100. Hay una **penalización por parámetros**:

Si su modelo tiene más de 20260 parámetros, la puntuación se reduce a la mitad.

El número de parámetros se mide mediante `sum(p.numel() for p in model.parameters())`. Se espera que el modelo también funcione en modo estocástico, con el `nn.Dropout` de PyTorch como parte del modelo.

Para las regiones estándar

Para cada región R (primera letra $\mathbb{I}$, $\mathbb{O}$, $\mathbb{A}$, `Background`), evaluamos el modelo en $N_R = 512$ puntos de prueba (x_i, y_i) con valores reales v_i y predicciones $\hat{v}_i$. Utilizamos el error absoluto medio (MAE) normalizado como métrica principal. El MAE se define como:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Y la normalización se realiza como

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

donde $s_R > 0$ es una constante de escala.

Para la región de la última letra $\mathbb{I}$

En esta región, **el dropout está habilitado durante la evaluación**. Para cada punto de prueba j :

1. Ejecutamos el modelo $K = 10$ veces para obtener $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Si alguna salida está fuera del rango $[-2026, 2026]$, entonces $\text{pointScore}(j) = 0$.
3. De lo contrario, calculamos la desviación estándar σ_j de las K salidas y la convertimos en una puntuación:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

donde $s_E > 0$ es una constante de escala fija.

La puntuación de la región es el promedio sobre todos los puntos de la región:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

donde $N_E = K * N_R$.

En términos sencillos, cuanto mayor sea la diversidad, mayor será la puntuación para esta región.

No se puede usar aleatoriedad en forma pura, incluidas las funciones `rand*` y `_uniform` de PyTorch; la aleatoriedad debe provenir de la inferencia con el dropout habilitado.

Cómo realizar la entrega

1. Abra `solution.ipynb` y ejecute todas las celdas.
2. Mejore el modelo `CustomModel` en `custom_model.py`
3. Asegúrese de que la última celda guarde el modelo en el archivo `model.pt`.
4. En la pestaña Git de JupyterLab, prepare, comente y confirme `solution.ipynb` y `custom_model.py`, y luego haga push.
5. Regrese a la página del concurso y haga clic en **Enviar**. El comentario de la entrega debe ser el mismo que el comentario del paso anterior.